

Seconde Option Sciences de l'ingénieur	TP 1 — Commande d'un moteur brushless Initiation à la programmation de systèmes embarqués Aéroglesseur	
Durée : 1 h 30	Outils : navigateur web (Chrome/Edge)	Matériel : ESP32-C3, aéroglesseur, ESC Dualsk

A	Prise en main — connexion BLE et procédure d'armement	20 min
B	Commande par curseur — relation commande / vitesse	25 min
C	Commande par joystick virtuel (NippleJS)	20 min
D	Bilan et questions	25 min

Contexte. L'aéroglesseur utilise un moteur **brushless** (sans balais) pour générer le coussin d'air qui le soulève. Ce moteur est piloté par un **ESC** (*Electronic Speed Controller*) qui traduit un signal numérique en courant triphasé. Dans ce TP, vous allez commander la vitesse de ce moteur depuis une page web, en communiquant sans fil par **Bluetooth Low Energy (BLE)** avec un microcontrôleur ESP32.

Objectifs du TP

- Réaliser la procédure d'armement et de calibration de l'ESC.
- Comprendre la relation entre la valeur numérique envoyée et la vitesse du moteur.
- Utiliser deux interfaces de commande différentes : curseur et joystick virtuel.
- Identifier l'influence des paramètres de l'ESC sur le comportement du moteur.

Rappels — Le moteur brushless et l'ESC

Le moteur brushless

Un moteur brushless est un moteur électrique **sans contacts glissants** (sans balais ni collecteur). Le rotor est un aimant permanent ; le stator porte trois bobines alimentées en courant triphasé. La rotation est obtenue en commutant électroniquement les bobines dans le bon ordre.

Avantages du brushless

Rendement élevé (> 85 %) — Longue durée de vie — Rapport puissance/masse excellent
— Pas d'étincelles

L'ESC et le signal PWM

L'ESC est le pont entre le microcontrôleur et le moteur. Il reçoit un signal **PWM** (*Pulse Width Modulation* — modulation de largeur d'impulsion) et génère en sortie le courant triphasé proportionnel à la consigne reçue.

Paramètre PWM	Valeur	Rôle
Fréquence	50 Hz (période = 20 ms)	Cadence des impulsions
Impulsion MIN	0,5 ms	Moteur à l'arrêt (0 %)
Impulsion MAX	2,5 ms	Pleine puissance (100 %)

Représentation d'un signal PWM à 50 % :

Calcul du rapport cyclique — duty_u16 (MicroPython)

L'ESP32-C3 représente le rapport cyclique sur 16 bits (valeur de 0 à 65 535).

$$\text{duty_u16} = \frac{t_{\text{impulsion}} \text{ (ms)}}{20 \text{ ms}} \times 65\,535$$

Impulsion 0,5 ms (arrêt) :	$\frac{0,5}{20} \times 65\,535$	= 1638
Impulsion 1,5 ms (mi-course) :	$\frac{1,5}{20} \times 65\,535$	= 4916
Impulsion 2,5 ms (pleine puissance) :	$\frac{2,5}{20} \times 65\,535$	= 8192

Partie A — Prise en main (20 min)

A1 — Préparation

Matériel à disposition :

- Aéroglesseur avec ESP32-C3 et ESC Dualsky XC-12-Lite
- Ordinateur ou tablette avec navigateur Chrome ou Edge
- Lien vers la page TP1 (GitHub Pages de votre fork) : <https://votre-pseudo.github.io/4webbluetooth/tp1>

Vérification préalable :

- ☐ L'ESP32 est alimenté — la LED NeoPixel est **rouge fixe**.
- ☐ La batterie de l'ESC est **débranchée**.
- ☐ La page HTML est ouverte en HTTPS (Web Bluetooth ne fonctionne pas en HTTP).

A2 — Connexion BLE

1. Cliquer sur **Connecter l'ESP32** dans la page web.
2. Dans la fenêtre de sélection, choisir **Aéroglesseur** (ou le nom configuré par votre binôme).
3. La LED passe au **bleu pâle** : la connexion est établie.

Sécurité — à lire avant de continuer

- Ne jamais approcher les doigts ou des objets de l'hélice lorsque le moteur tourne.
- Ne jamais laisser l'aéroglesseur sans surveillance lorsque la batterie est connectée.
- En cas de problème, utiliser le bouton **ARRÊT D'URGENCE** de la page web ou débrancher immédiatement la batterie.

A3 — Procédure de calibration et d'armement de l'ESC

La première utilisation nécessite d'apprendre à l'ESC les valeurs MIN et MAX du signal PWM. Suivre les étapes affichées dans la page web :

Étape	Action	Signal sonore ESC	LED ESP32
1	Mettre le curseur à 100 %	—	Orange
2	Connecter la batterie	Mélodie + bips cellules	Orange
3	Entendre le long bip (MAX détecté)	1 long bip	Bleu
4	Cliquer — throttle passe à 0 %	—	Violet
5	Entendre la mélodie (MIN + armé)	Mélodie	Vert

Deuxièmes utilisations (démarrage rapide)

Si l'ESC est déjà calibré, la procédure simplifiée suffit : throttle à **0 %** → connecter la batterie → attendre les bips de comptage cellules → ESC armé.

Q1. Combien de bips entendez-vous lors de l'étape 2 ? Que signifient-ils ?

Partie B — Commande par curseur (25 min)

B1 — Prise de mesures

Une fois l'ESC armé (LED verte), la section **Commande des gaz** apparaît. S'assurer que le mode **Curseur** est sélectionné.

Déplacer progressivement le curseur et observer le comportement du moteur. Relever dans le tableau ci-dessous la valeur du curseur à laquelle :

Observation	Valeur du curseur (%)
Le moteur commence à tourner (seuil de démarrage)	
Le moteur tourne à vitesse modérée, régulière	
Le moteur atteint sa vitesse maximale	

Q2. La relation entre la valeur du curseur et la vitesse de rotation vous semble-t-elle linéaire ? Justifier qualitativement.

B2 — Le signal PWM vu par l'ESP32

Q3. En vous aidant du tableau de rappel (page 1), calculer la valeur `duty_u16` correspondant à une impulsion de **1,0 ms** (position de départ avant calibration étendue).

Q4. Avec la plage étendue (0,5 ms à 2,5 ms), combien de niveaux de vitesse distincts peut-on théoriquement représenter ? Comparer avec une plage standard (1,0 ms à 2,0 ms).

B3 — Extrait du code MicroPython

Voici la fonction de conversion utilisée dans le firmware de l'ESP32 :

```
aeroglisser_ble.py — fonction set_throttle

ESC_MIN = 1638    # duty_u16 pour 0,5 ms
ESC_MAX = 8192    # duty_u16 pour 2,5 ms

def set_throttle(pct):
    """pct : entier 0-100"""
    duty = int(ESC_MIN + (ESC_MAX - ESC_MIN) * pct / 100)
    esc.duty_u16(duty)
```

Q5. Calculer la valeur duty renvoyée par `set_throttle(50)`. Vérifier que cela correspond à une impulsion de 1,5 ms.

Partie C — Commande par joystick virtuel (20 min)

C1 — Basculer en mode Joystick

Cliquer sur le bouton **Joystick** dans la page web. Un cercle représentant un joystick virtuel apparaît (bibliothèque **NippleJS**).

C2 — Prise en main

Faire glisser le joystick vers le **haut** et observer :

- La valeur numérique affichée sous le joystick.
- Le comportement du moteur.
- Ce qui se passe quand on relâche le joystick.

Q6. Quelle différence de comportement observez-vous entre le curseur (partie B) et le joystick (partie C) lorsque vous relâchez le contrôle ?

Q7. Dans quel sens doit-on déplacer le joystick pour accélérer ? Proposer une explication physique de ce choix de conception.

C3 — Extrait JavaScript

Voici comment la page web convertit la position du joystick en commande BLE :

tp1_moteur.html — gestion du joystick NippleJS

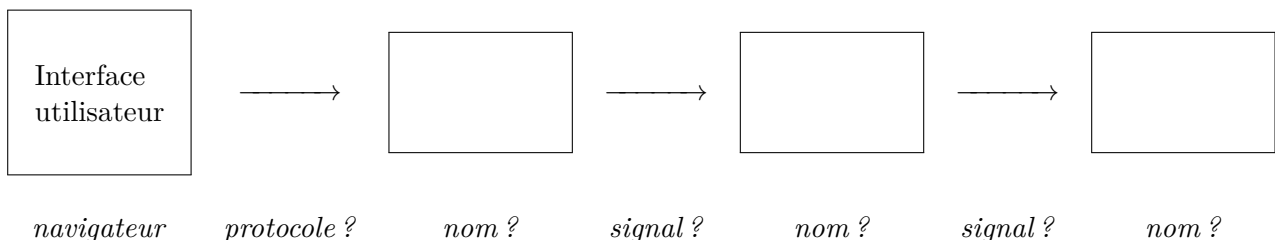
```
manager.on('move', (evt, data) => {  
  const y = data.vector.y;           // valeur entre -1 et +1  
  const throttle = Math.max(0, y) * 100;  
  envoyerThrottle(throttle);  
});  
  
manager.on('end', () => {  
  envoyerThrottle(0);                // retour à 0 au relâchement  
});
```

Q8. Pourquoi utilise-t-on `Math.max(0, y)` plutôt que `y` directement ?

Partie D — Bilan (25 min)

D1 — Schéma fonctionnel

Compléter le schéma de la chaîne de commande en plaçant les éléments suivants dans les cases : *ESP32-C3*, *navigateur web*, *ESC*, *moteur brushless*, *BLE*, *PWM (50 Hz)*, *signal numérique (0–100 %)*.



D2 — Questions de synthèse

Q9. Quel est le rôle du protocole BLE dans cette chaîne ? Pourquoi ne pas utiliser le Wi-Fi ?

Q10. En cas de déconnexion Bluetooth inattendue, que se passe-t-il sur le moteur ? Comment est-ce géré dans le firmware ? (Chercher dans `aeroglisseur_ble.py`, événement `_IRQ_CENTRAL_DISCONNECT`.)

Q11 (approfondissement). Le paramètre `INTERVALLE_MIN = 50 ms` est défini dans la page HTML. À quelle fréquence maximale les commandes BLE sont-elles envoyées ? Pourquoi limiter cette fréquence ?

Compte rendu — à remettre en fin de séance

- Tableau de mesures complété (B1).
- Réponses aux questions Q1 à Q11.
- Schéma fonctionnel complété (D1).